

1) O que é montar um programa?

R: No sentido de AOC, montar um programa seria a parte de traduzir um código de alto nível como o C para uma linguagem de montagem que depois iria para uma linguagem de máquina que o computador seria capaz de executar e realizar as operações desejadas.

2) Qual é a diferença entre Arquitetura e Organização de um processador? Qual é mais abstrato?

R: Enquanto a arquitetura é menos abstrata e foca mais no que está perto do programador, a parte organizacional é mais próxima do hardware e demanda mais compreensão de projeto do que somente a arquitetura.

3) Uma arquitetura pode ter várias organizações? Por quê?

R: Sim, uma arquitetura pode ter várias organizações diferentes para também pode atender a demandas diferentes. Ou seja, o propósito do programa é o mesmo, mas a forma com que o hardware lida com aquela informação pode ser completamente diferente. Dando saídas iguais mas passando por processos diferentes

4) Quais são os principais atributos arquiteturais de um processador?

R: Tamanho da palavra de dados, tipos de dados, tamanho da palavra de instrução, formato das instruções, modos de endereçamento, registradores, conjunto de instruções.

5) Qual o formato genérico de uma instrução e como as suas partes são definidas?

R: Segue a lógica [opcode | operando]

6) O que forma um programa?

R: A lógica que forma um programa é basicamente instruções com operandos, entradas e saídas da memória, organizados em forma lógica e sequencial afim de resolver um problema

7) Quais os componentes que formam a Unidade Central de Processamento (UCP ou CPU)? Explique-os.

R: Memória principal
Unidade Lógica de processamento
Registradores

8) Qual é a função do apontador de instrução (PC)?

R: O Program Counter serve para indicar o endereço na memória da instrução a ser executada. É incrementado a cada instrução executada para que o programa avance para a instrução seguinte.

9) Qual a relação que o PC tem com o tamanho da memória de programa?

R: O número de bits do program counter determina a quantidade máxima de endereços máxima que a memória do programa pode ter. Sempre seguindo a regra de 2^N , sendo N o número de bits de largura.

10) O que é utilizado pela Unidade de Controle para gerar os sinais de controle?

R: É utilizado os bits de controle da sequência de bits recebidas, o opcode informa a unidade de controle o que deve ser feito de instrução na unidade de processamento

11) Explique o processo de “busca - decodificação - execução” de uma instrução?

R: Esse é o processo executado pela CPU para processar uma informação recebida, decodificar o que deve ser feito e instruir o processador sobre o que fazer com aqueles dados recebidos. É um ciclo repetitivo até o final da execução do programa

12) Quais as três classes de instruções do BIP? Dê exemplos.

Instruções de controle -> Paralisar a operação

instruções de aritmética -> Adicionar, subtrair

instruções de transferência -> Salvar dados

13) Quais os tamanhos das palavras do BIP e suas interpretações?

R: São 16 bits, sendo os bits de 15 a 11 os de operação e os de 10 a 0 são os que definem o operando

14) Quais as diferenças entre os modos de endereçamento imediato e direto na visão arquitetural do BIP?

R: No modo imediato, o próprio valor do dado que você quer usar já está embutido dentro da própria instrução (no campo do operando), exige um único acesso à memória.

No modo direto, o dado a ser processado é uma variável na memória apontada pelo operando da instrução

15) Sabendo que A=0x51 (), B=0x54 (), C=0x56 () e seguindo a lógica de opcode para BIP I

- Lembrando que opcode é 5 bits

Endereço	Conteúdo: hexa == binário
0	1802 0001 1000 0000 0010
1	0851 0000 1000 0101 0001
2	1801 0001 1000 0000 0001
3	0854 0000 1000 0101 0100
4	1051 0001 0000 0101 0001
5	3054 0011 0000 0101 0100
6	3801 0011 1000 0000 0001
7	2803 0010 1000 0000 0011
8	0856 0000 1000 0101 0110
9	0000 0000 0000 0000 0000

16) ACC = 3 | PC = 9 | 0x51 = 2 | 0x54 = 1 | 0x56 = 1

18) Faça um programa no BIP que copie os últimos 3 números da sua matrícula separadamente para os endereços 0, 1 e 2 da memória de dados. Depois, some 1 a cada um desses números e guarde na posição 3, 4 e 5. Finalmente, monte o código em linguagem de máquina.

Código assembly com 637:

START:

// Armazenando a matrícula sequencialmente de em 0,1,2

```
LDI 6      ; ACC <= 6
STO A      ; (A) <= ACC
LDI 3      ; ACC <= 11
STO B      ; (B) <= ACC
LDI 7      ; ACC <= 11
STO C      ; (C) <= ACC
```

// Fazendo a soma e armazenando

```
LD A       ; ACC <= (A)
ADDI 1     ; ACC <= ACC+1
STO D      ; (D) <= ACC
```

```
LD B       ; ACC <= (B)
ADDI 1     ; ACC <= ACC+1
STO E      ; (E) <= ACC
```

```
LD C       ; ACC <= (A)
ADDI 1     ; ACC <= ACC+1
STO F      ; (D) <= ACC
```

END

HLT

Endereço	Conteúdo: hexa == binário
0	3806 == 0011 1000 0000 0110
1	1000 == 0001 0000 0000 0000
2	3803 == 0011 1000 0000 0011
3	1001 == 0001 0000 0000 0001
4	3807 == 0011 1000 0000 0111
5	1002 == 0001 0000 0000 0010
7	2000 == 0010 0000 0000 0000
7	5801 == 0101 1000 0000 0001
8	1003 == 0001 0000 0000 0011
9	2001 == 0010 0000 0000 0001
10	5801 == 0101 1000 0000 0001
11	1004 == 0001 0000 0000 0100
12	2002 == 0010 0000 0000 0010
13	5801 == 0101 1000 0000 0001
14	1005 == 0001 0000 0000 0101
15	0000 == 0000 0000 0000 0000